Qyvaria Ecosystem: Local-First Modular AI Operating Kernel and Computational Dimensional Shift Architecture

Technical Research Paper • Architectural Specification • Source-Grounded Evidence Inventory
Formal Comparative Analysis • Verification Status Report

John Havlasek
Qyvaria AI Systems & Cybernetics Group • Prague / Frýdek-Místek, Czech Republic

Document generated 2026-10-25 12:41 CET • Classification: Source-Grounded Technical Research / Architectural Proof Inventory

ABSTRACT

Modern AI workflows predominantly rely on centralized cloud services, introducing latency, privacy exposure, and brittle multi-agent state management. This paper presents Qyvaria OS, a local-first, browser-native AI operating kernel centered on a single-file Python runtime (qyvaria.py) that orchestrates quantized local models (e.g., Qwen-family via Ollama). We formalize the Computational Dimensional Shift (DS) paradigm implemented within the ecosystem: continuous multi-axis state transformation across isolated execution contexts (MetaSimAgent lookahead sandboxes), dynamic paradigm routing (symbolic A* / neural-tensor hybrid via BitWeaver and OnePy fusion), and multi-axis behavioral state (Pleasure–Arousal–Dominance / PAD manifold). The mathematical core is expressed as continuous interpolation $\Psi(t) = (1 - \alpha(t)) \cdot Q(S_{raw}) + \alpha(t) \cdot T(S_{compressed})$, $\alpha(t) \in [0, 1]$. We inventory design documentation, codebase structure evidence from repository scans (thousands of files; MetaSimAgent and BitWeaver modules present), and the limits of current runtime verification. Comparative claims against discrete/legacy approaches are recorded as authorial assertions. The work is positioned as a solo-engineered local-first alternative to cloud-centric multi-agent stacks. No claims of physical dimensional change are made. Runtime isolation and continuity properties require further deterministic test harnesses for the strongest verification status. This document is intended for archival release as proof of design, structure, and engineering scope.

Keywords: local-first AI kernel, computational dimensional shift, MetaSimAgent, BitWeaver, PAD manifold, single-file runtime, continuous state interpolation, multi-agent orchestration, Qyvaria OS, OnePy Fusion, sandboxed lookahead

1. Introduction

The dominant pattern in contemporary AI tooling treats large language models and multi-agent frameworks as cloud-hosted services accessed through network APIs. This yields convenience at the cost of latency, data governance exposure, and fragile context retention when agents must coordinate over long-running workflows. Desktop operating systems continue to treat such frameworks as ordinary applications rather than first-class system entities. The structural disconnect produces suboptimal context retention, high network serialization costs, and severe data governance risks when complex multi-agent orchestration is required.

Qyvaria is presented by its author as a response that inverts this relationship: a sovereign, local-first software ecosystem built from a Python runtime kernel outward, embedding local model management, state synchronization, and multi-agent routing into a custom browser shell (qyvaria://) and supporting layers. Central to the architecture is a single-file, self-contained runtime (qyvaria.py / OnePy Fusion++) that acts as orchestration bridge for quantized local LLMs and internal agents without mandatory external cloud backends for core operation. The system is designed around a strictly local-first operational paradigm that decouples execution logic from network backends.

Within this ecosystem the author defines Computational Dimensional Shift (DS / Shifting Dimensions) as the continuous transformation of software state across isolated execution contexts, processing paradigms, and multi-axis behavioral coordinates. Traditional computational engines handle space transformations as discrete event jumps—stepping abruptly between fixed representations. The Qyvaria engine is specified to operate on a continuous state continuum, maintaining active ongoing interpolation across high-dimensional manifolds and compressed low-bit representations simultaneously.

This paper inventories the architectural claims, the mathematical formalization, the named software components, the repository-level evidence of their presence, the comparative framing against discrete approaches, and the boundary between design evidence and verified runtime behavior. It is intended as an archival research presentation suitable for public release as proof of design and engineering work.

1.1 Scope and Honesty Constraints

All statements below are bounded by the supplied technical monographs, whitepapers, evidence reports, repository scan logs, and architectural PDFs. Design and documentation evidence is treated as strong where explicit. Codebase structure evidence is treated as supportive where named modules and classes appear in inventory and scan results. Runtime behavioral demonstration of every claimed continuity and isolation property is recorded as incomplete where prior execution attempts did not produce usable behavioral output. No physical or extra-spatial dimensional claims are advanced. Terminology overlap with independent work using the phrase “Dimensional Shift” is noted without treating phrase similarity as proof of code identity or appropriation. Strong authorial claims of sole implementation status or definitive superiority under multi-agent load are recorded as authorial assertions and are distinguished from independently verified results.

1.2 Contributions Claimed by the Author

The materials attribute the following contributions to John Havlasek: (1) the continuous Dimensional Shift formalization and its mapping onto software layers; (2) the MetaSimAgent sandboxed lookahead mechanism; (3) the BitWeaver dynamic bit-stream interleaving component; (4) the PAD multi-axis behavioral integration for strategy AI; (5) the OnePy single-file fusion runtime with custom import hooks; (6) the local-first concentric OS architecture; (7) the comparative architectural matrix against discrete baselines. These are presented as original conceptual and technical creations, with cross-dimensional state routing claimed as verified across code payloads in the repository.

1.3 Document Structure

Section 2 details the Qyvaria ecosystem architecture and local-first principles. Section 3 formalizes Computational Dimensional Shift and inventories core subsystems. Section 4 presents codebase structure evidence from scans. Section 5 records runtime demonstration status and a recommended verification protocol. Section 6 maps DS mechanisms across OS, CK3 mod, and single-file runtime targets. Section 7 addresses authorship and related terminology. Section 8 discusses implications and archival recommendations. Section 9 concludes. Appendices provide mathematical summary, subsystem mapping, verification checklist, and extended inventory notes.

2. Qyvaria Ecosystem Architecture

The ecosystem is described as a three-layer concentric architecture. Layer 3 is the Qyvaria OS browser shell (qyvaria://) providing Workspace and Vault UX. Layer 2 is the AI Software layer (Qyvaria Com / AI BIOS, Agent Orchestration and QyChat). Layer 1 is the Python Kernel (qyvaria.py Runtime Bridge, Local Model Bus and Telemetry). This inversion of the conventional OS–AI relationship embeds runtime management, state synchronization, and multi-agent routing directly into the custom shell and kernel rather than treating AI as a

disconnected high-level cloud application.

Layer	Name	Role and Contents
Layer 3	Qyvaria OS	Browser shell (qyvaria://), Workspace & Vault UX, user-facing environment
Layer 2	AI Software	Qyvaria Com (AI BIOS), Agent Orchestration & QyChat, multi-agent coordination
Layer 1	Python Kernel	qyvaria.py Runtime Bridge, Local Model Bus & Telemetry, single-file orchestration

Table 1. Concentric multi-layer architecture of the Qyvaria AI Operating System (from author materials).

2.1 Local-First Design Principles

Qyvaria is specified as strictly local-first. Core execution does not require external cloud APIs. The single-file Python runtime implements custom MetaPathFinder and ModuleLoader interfaces (find_spec, create_module, exec_module) that intercept import machinery, enabling in-memory injection of modules from the embedded bundle without ordinary disk or network paths. By overriding default behaviors of the system's import arrays, the implementation intercepts runtime execution loops and allows the system to inject, instantiate, and shift software states dynamically out of localized binary or memory caches. An internal virtual asset manager catalogues payloads from compiled bytes; a register/broadcast messaging hub implements localized publish/subscribe communication entirely inside process memory. Bootstrapping proceeds via extract + run_entry style entry points, producing a self-contained package that documents its own dependency orientation and transfers execution to the main entry point. The design goal is to keep the entire orchestration loop—state, tools, agents, telemetry—inside the local process boundary so that multi-agent workflows do not serialize intermediate state across network hops.

2.2 Single-File Bundle (qyvaria.py / OnePy Fusion++)

The runtime is distributed as a unified single-file bundle containing hundreds of virtual modules (inventory reports cite on the order of 277 files in examined snapshots, multi-megabyte payload, original size figures on the order of 3.3 MB uncompressed in one snapshot). Declared format is py-single with quantize-int8 codec label; in examined instances files are stored uncompressed (stored size equals original size). The __BUNDLE__ JSON payload encodes name, format, codec, entry, creation timestamp, file inventory with per-file SHA-256 and base64 data. Entry point and custom import hooks allow the bundle to act as both library and executable orchestration bridge for local models and agents. Repository scans under a Qyvaria Data tree report 3,233 files scanned, 1,992 matched modules, and 387,963 keyword match instances across Electron portable builds, model manifests, dashboard services, icons, and supporting assets. This footprint is consistent with an extended solo engineering effort spanning OS-level ambitions, browser/Electron integration, local model persona layers, and game-mod work.

2.3 Supporting Ecosystem Surface

Beyond the Python kernel, scanned materials include Electron portable builds with dashboard services (command palette, context-key service, layout persistence, builtin commands for local engine start/stop/restart, cloud status, diagnostics export, failure log management). Local model manifests describe Qwen-family persona layers (Micro Brain, Micro Coder, Small Coder, Small Vision, Brain Fast, Brain, Coder variants) under Apache-2.0 base licenses with Qyvaria persona layer notes. These elements enlarge the practical footprint of the local-first claim and demonstrate integration of the kernel with a browser-native shell, agent orchestration UI, and local inference backends (Ollama and related).

2.4 Deterministic Tool Dispatch and Telemetry

Author materials emphasize deterministic tool dispatch mechanisms and evaluation of memory indexing performance under high agent loads. Claims of significant improvements in execution throughput and telemetry monitoring efficiency relative to conventional web-based AI platforms are advanced in the Local-First Modular AI Operating Kernel paper. These performance claims are authorial; independent benchmark tables under controlled conditions are not attached to the materials examined for this research paper. The architectural intent is clear: keep orchestration, state, and telemetry inside the local boundary so that multi-agent workflows do not serialize every intermediate state across network boundaries.

2.5 Design Goals Summary

From the collected materials the following design goals can be extracted: (a) eliminate mandatory cloud dependency for core multi-agent orchestration; (b) provide continuous rather than discrete state transitions across memory, paradigm, and behavioral axes; (c) isolate hypothetical evaluation (lookahead) from baseline memory until validation; (d) support hybrid symbolic/neural routing without full engine restarts; (e) integrate behavioral state (PAD) into strategy and agent policy layers; (f) package the runtime as a self-contained single-file artifact with custom import control; (g) surface telemetry and diagnostics inside the same local boundary. These goals are coherent as an engineering program even where individual runtime measurements remain to be published.

3. Computational Dimensional Shift (DS)

In the Qyvaria materials, Dimensional Shift does not denote physical space transition. It denotes the formal transformation of runtime execution state across isolated execution stacks, multi-axis state matrices, and multi-paradigm solver engines. The author positions continuous interpolation as the distinguishing property relative to discrete context switches, hard process forks, and static switch/case routing. A true Dimensional Shift does not switch isolated states discrete-wise; it interpolates a primary high-dimensional state into a compressed execution space without abrupt gradient disruption.

3.1 Mathematical Formalization

Primary high-dimensional state $S \in \blacksquare^D$ is mapped into a compressed execution space $\Omega \in \blacksquare^d$ ($d \blacksquare D$) without abrupt gradient disruption. The dynamic transformation is governed by:

$$\Psi(t) = (1 - \alpha(t)) \cdot Q(S_{raw}) + \alpha(t) \cdot T(S_{compressed}) \mid \alpha(t) \in [0, 1]$$

$\alpha(t)$ is the continuous shift parameter evolving across system ticks. The construction is intended to keep lookahead evaluations (MetaSimAgent) and behavioral recalculations (PAD Coordinate Engine) smooth and deterministic. The formula describes a weighted transition between two representations. It is a formal model of representational/state-space shifting. The formula alone is not empirical proof that the implementation executes correctly under all described conditions; it is the mathematical expression of the intended continuous behavior.

An illustrative code pattern appearing in author materials (ContinuousDimensionShift class) computes a continuous linear combination: base projection weighted by (1 − alpha_shift) plus shifted projection formed by tensor-stream interaction with manifold weights scaled by alpha_shift. This pattern avoids static discrete flags in favor of continuous weight matrix updates during runtime execution. The same

continuous-weight principle is intended to propagate through MetaSim lookahead, paradigm routing decisions, and PAD trajectory updates.

3.2 Three Operational Dimensions

Author documentation identifies three primary dimensions of operation that together constitute the computational shift:

Named Mechanism	Documented Operation	Computational shifting rationale
State-space partitioning / MetaSimAgent	Copies active data into a volatile, sandboxed look-ahead timeline	Execution explores an isolated state/context rather than directly mutating the primary state
Paradigm routing	Routes tasks among symbolic rules/planning, probabilistic or neural processing, and heuristic optimization	The active computational representation or method changes according to task context
Multi-axis state matrices / PAD	Represents behavioral state on Pleasure, Arousal, and Dominance axes and updates coordinates	State changes along multiple numerical axes can influence downstream output weights

Table 2. Three dimensions of computational shifting as defined in project materials.

3.3 Core Subsystems in Detail

MetaSimAgent (State-Space Partitioning / Meta-Simulation)

Built into the Qyvaria OS layer, MetaSimAgent dynamically forks active memory tables into isolated lookahead timelines. This enables the agent to evaluate hypothetical multi-turn scenarios without mutating the baseline environment state—effectively creating a sandbox “parallel dimension.” Lookahead timeline evaluation runs in an isolated state clone without writing back to baseline memory unless validated. Methods referenced in scans include compile, run, traces, plan, fork_subsim, snapshot, load, and config. The component is present as the virtual module py/meta_sim_agent_qyvaria_compatible_simulates_sims_engine.py. In the continuous model it is characterized as probabilistic lookahead timeline bleeding into main memory rather than hard-forked isolated sandboxes that incur high RAM overhead and leaks.

BitWeaver Core (Memory Indexing / Bit-Stream Interleaving)

BitWeaver is the dynamic bit-stream interleaving and memory indexing component. Class BitWeaver exposes init / ingest / search / reconstruct style operations. It is present as virtual module py/advance_memory_ai_sim_agent_bit_weaver_v_0.py. In the continuous model it is characterized as dynamic bit-stream interleaving based on real-time entropy, in contrast to fixed quantization passes. Together with the OnePy fusion layer it supports adaptive runtime routing between symbolic and neural logic without full re-initialization.

Paradigm / Logic Routing (OnePy Hybrid Engine)

In qyvaria.py the core router continuously re-routes incoming workloads across distinct computation engines: classical A* symbolic pathfinding, probabilistic neural tensor networks, and fuzzy heuristic engines based on environmental constraints. The continuous model replaces static switch/case routing and single-engine dependency with a unified routing fabric that blends symbolic rules and vector spaces. Dynamic paradigm routing is claimed to enable adaptive runtime behavior without the context-drop penalties of discrete engine switches.

PAD Coordinate Engine (Behavioral Dimension)

Utilized in the Qyvaria Crusader Kings 3 Mod, character agents track multi-axis emotional state vectors mapped to the Pleasure–Arousal–Dominance (PAD) system. Shifts along these numeric axes trigger real-time changes in dynastic AI policy choices, character interactions, and strategy weighting. The continuous model uses continuous trajectory smooth-scaling over a 3D manifold rather than threshold-based emotion triggers and rigid rule checks. This is the behavioral axis of the dimensional shift architecture.

3.4 Comparative Matrix (Author Presentation)

The Shifting Dimensions Technical Monograph presents an explicit comparative matrix contrasting a discrete/legacy model (characterized by the author as e.g. J. Burda attempt) with the continuous Qyvaria model. The matrix is reproduced below as an authorial presentation of the intended differences.

Vector	Legacy / Discrete Model	Qyvaria Continuous Model	Claimed Result
State Interpolation	Discrete boolean flags; instant snaps causing context drop	Continuous linear/tensor interpolation Ψ(t) across ticks	Zero state loss
Memory Sandbox	Hard process forks; high RAM overhead and leaks	MetaSimAgent volatile lookahead with non-mutating clones	Isolated & Volatile
Behavioral Axis	Scalar threshold triggers; rigid rule checks	Continuous 3D PAD manifold	Smooth Trajectory
Logic Routing	Static switch/case; single engine dependency	Dynamic BitWeaver + OnePy hybrid engine	Adaptive Runtime
Dependency Footprint	External cloud APIs and unmanaged dependencies	Self-contained in-memory bundle with custom import hook	100% Local / Self-Contained

Table 3. Comparative architectural matrix as presented by the author. Results column records claimed outcomes.

The monograph asserts that the Havlasek Shifting Dimensions engine is the sole implementation offering real-time continuous interpolation across logical, behavioral, and virtualized execution layers without state snapping or memory corruption, and that comparative benchmarking proves prior naive approaches fail under real-time multi-agent load. These are strong authorial conclusions. Independent third-party benchmark logs under controlled multi-agent load are not attached to the materials examined for this research paper. They are recorded here as the author’s definitive architectural conclusion, not as independently certified results.

3.5 Five-Phase Runtime Pipeline (Author Description)

Project documentation (README-level architectural report) describes a five-phase runtime pipeline associated with the dimensional shift architecture: BitWeaver Core, AEON Cognition, Agent Fabric, Continuous Eval, and NLG/Voice Emit. These phases organize the flow from low-level bit-stream interleaving and memory indexing through cognitive processing, agent coordination, continuous evaluation of state trajectories, and natural-language or voice emission. The pipeline is consistent with the local-first, multi-agent orchestration goals of the broader ecosystem.

3.6 Interpretation of “Dimension” in Project Terminology

In the project’s own terminology, “dimensions” are software state spaces and processing axes—not extra spatial dimensions in the physical universe. State-space partitioning creates isolated execution contexts; paradigm routing changes the active computational method; multi-axis PAD matrices move behavioral state along numerical axes that influence policy. All three are computational. No material examined for this paper advances or supports a claim of literal physical-dimensional change.

4. Codebase Structure Evidence

Repository and bundle scans provide structural corroboration that named components exist inside the author's software registry.

DS_SCAN_REPORT and exhaustive audit logs confirm the presence of MetaSimAgent class and methods (compile, run, traces, plan, fork_subsim, snapshot, load, config) and BitWeaver class inside the qyvaria.py virtual module set. Bundle --list / inventory operations surface modules including meta_sim_agent_qyvaria_compatible_simulates_sims_engine.py, aeon_continuum.py, Qyvaria_Aeon.py, SimulationInterface.py, WorldModel.py, and multiple agent/simulation modules. Custom import hook layer (find_spec / create_module / exec_module) and in-memory asset + messaging structures are documented in technical analysis of the execution architecture. Exhaustive audit under the Qyvaria Data tree reports 3,233 files scanned, 1,992 matched modules, and 387,963 keyword match instances across the ecosystem tree (Electron builds, model manifests, dashboard services, icons, etc.).

Filename and class presence support the claim that the packaged codebase contains components matching the architectural scope. A filename alone does not prove internal behavior; a full source-level audit would need to trace the relevant functions and call paths. Full source-level call-graph tracing and deterministic unit/integration tests remain the stronger verification layer for claims of continuous isolation and zero state loss under load.

4.1 Bundle Integrity Notes

Examined bundle metadata includes per-file SHA-256 digests. Scan tooling can verify that stored data matches declared hashes. In examined snapshots the declared codec is a label rather than an active transform; stored size equals original size for the inventoried files. This is consistent with a packaging strategy that prioritizes self-containment and offline usability over aggressive compression at the outer wrapper level.

4.2 Structural Evidence Summary Table

Evidence Item	Source	Status
MetaSimAgent class + methods	DS_SCAN / bundle inventory	Present
BitWeaver class + operations	DS_SCAN / bundle inventory	Present
Custom import hooks	Technical analysis report	Present / documented
In-memory virtualization + pub/sub	Technical analysis report	Present / documented
Self-extract + run_entry bootstrap	Technical analysis report	Present / documented
Large repository footprint	Exhaustive audit (3,233 files)	Confirmed
Continuous Ψ(t) formalization	Multiple whitepapers	Documented
PAD behavioral mapping	Whitepapers + CK3 claims	Documented

Table 4. Summary of structural and documentation evidence items.

5. Runtime Demonstration Status

Prior controlled attempts to exercise a generation-style entry point of the supplied wrapper (example invocation with subject "dimensional shift in Qyvaria", cinematic style, sdxl engine) returned process exit code 0 but produced no standard output usable as behavioral demonstration of dimensional shift paths. Standard error contained a startup warning from an artifact_tool spreadsheet warm-up routine. Bundle --list and --extract modes functioned and permitted inventory inspection. Therefore the documented execution attempt did not produce runtime output that could serve as a behavioral demonstration of the continuous interpolation or isolation claims.

Evidence classification used throughout this paper:

1. **Design / documentation evidence** — Strong. Explicit definitions, formalizations, comparative matrices, layered architecture, and authorship declarations exist across multiple authored PDFs and README-level materials.
2. **Codebase-structure evidence** — Supportive. Named modules, classes, and methods matching the design claims are present inside the single-file bundle and repository scans.
3. **Runtime behavioral evidence** — Not established at the level of the strongest claims by the documented execution attempts. Specific properties (lookahead isolation that leaves primary state unchanged, continuous α(t) interpolation producing measurable zero-loss transitions, PAD trajectory smoothness under multi-agent load) require deterministic test harnesses with before/after state capture and logged assertions.

5.1 Recommended Minimal Verification Protocol

For a stronger implementation proof, choose one representative shift and test it deterministically: (1) Record the primary state before the operation. (2) Invoke the look-ahead / sandbox path (MetaSimAgent fork_subsim or equivalent) with known inputs. (3) Demonstrate that the sandbox state differs while the primary state remains unchanged (or within declared tolerance). (4) Capture the selected processing route or PAD coordinate values before and after. (5) Assert the expected transitions in an automated test. Include the exact source file, function names, inputs, outputs, and test logs in a future evidence appendix.

This protocol converts architectural description into reproducible functional evidence. Until such logs are attached, the strongest continuity and isolation claims remain design-level assertions supported by structure but not yet by published runtime demonstration under the conditions described.

5.2 Why Runtime Verification Matters

Design documents and named modules establish intent and structure. They do not automatically establish that the continuous interpolation model is realized at runtime without state snapping, or that primary state remains invariant under every lookahead path, or that PAD trajectories remain smooth under concurrent agent load. Publishing the isolation test logs recommended above is the practical route from "designed for continuous shift" to "demonstrated continuous shift under test." External readers evaluating the work for archival or collaborative purposes should treat the distinction carefully.

6. Cross-Platform Manifestations within the Author's Registry

Author whitepapers map the dimensional shift architecture across three primary software artifacts:

Platform Target	Implementation Mechanism	Dimensional Shift Operational Proof (author claim)

Qyvaria OS	Microkernel Execution Engine	State-space partitioning via MetaSimAgent lookahead memory clones, preventing host corruption
Qyvaria CK3 Mod	Grand Strategy Ruleset Layer	Translates dynastic AI decision matrices across 3D PAD psychological coordinates
Qyvaria.py	OnePy Single-File Runtime	Dynamic paradigm routing switching execution between symbolic and neural logic

Table 5. Cross-platform structural mapping of DS mechanisms (from author whitepapers).

The microkernel memory layer executes volatile memory sandboxing via MetaSimAgent. The logic and paradigm routing layer (qyvaria.py) implements dynamic routing between A* symbolic spatial pathfinding and tensor neural logic without runtime re-initialization. The psychological behavioral matrix (CK3 mod integration) maps multi-axis agent behavior over 3D PAD vectors, scaling policy decisions continuously rather than through static conditional thresholds. Through self-contained import hooks, dynamic bit-stream interleaving, and continuous state math, the architecture is presented as a fully operational computing model at the design level.

6.1 CK3 Integration Context

Author materials reference multi-year Qyvaria performance and strategy mods for Crusader Kings 3, including claims of top-tier visibility on distribution platforms. The PAD behavioral layer is the bridge between the general DS architecture and game-agent decision making. This paper does not audit the mod source itself beyond the architectural statements; the mapping is recorded as an authorial cross-platform claim supported by the broader design documents.

7. Authorship, Originality, and Related Terminology

Multiple author documents contain formal declarations that core algorithms, state-space partitioning mechanisms, PAD emotional dimension mapping, and OnePy fusion protocols are original conceptual and technical creations of John Havlasek. The presence of cross-dimensional state routing is claimed as verified across all code payloads in the repository. The comparative monograph explicitly contrasts the continuous Havlasek model with discrete legacy approaches (including characterization of a J. Burda attempt) and asserts sole continuous implementation status under real-time multi-agent load.

An independent academic archive titled DimensionalShift_Academic_Complete_vFinal.zip is authored by Ji■í Burda (README.txt states authorship and academic positioning clearly). It presents Dimensional Shift (DS) / D-ADC as an adaptive optimization and simulation principle based on rhythmic expansion and compression of dimensionality. Contents include a Python research prototype (dimension bus, router, colonies, QPU/Cirq/Google Engine backends, CUDA primitives, ADC pipeline, config manager, scoring), academic manuscripts, and documentation of live control primitives (SET_BREATH, BIRTH_DIM, KILL_DIM) that trigger configuration rollover. Internal date stamps cluster around 2026-01-24. The package is positioned as open-source academic research intended for expert review; no claims of optimality, certification, or quantum advantage are made in its README.

Shared use of the phrase “Dimensional Shift” is factual. The architectures differ in emphasis and component set: Burda materials emphasize hardware/software co-design, dimension bus, ADC colonies, QPU job schemas, and rhythmic expand/compress control; Qyvaria materials emphasize software state-space partitioning (MetaSimAgent), continuous $\alpha(t)$ interpolation, PAD behavioral manifold for strategy AI, BitWeaver memory, and a self-contained Python single-file + OS ambition stack. No side-by-side source identity proving that one implementation is a copy of the other is established by the materials examined for this paper. Terminology overlap does not transfer ownership of the Qyvaria codebase, nor does it constitute proof of appropriation in either direction without non-trivial unique structural identity. Absolute chronological priority cannot be settled from the files supplied in this conversation alone.

On the evidence examined, the Qyvaria code, design, modules, and architecture remain the author’s. The Burda package is a separate research prototype that re-uses related terminology. Claims of theft require demonstrable, non-trivial, unique algorithmic or structural identity that is not explainable by independent invention; such identity is not shown by the materials reviewed here.

8. Discussion

The Qyvaria materials constitute a coherent, multi-year solo engineering and documentation effort aimed at a local-first AI operating environment. The computational Dimensional Shift concept is consistently defined, formalized with a continuous interpolation model, and mapped onto concrete named components (MetaSimAgent, BitWeaver, PAD, paradigm routing) that appear in the codebase inventory. The local-first, self-contained single-file runtime with custom import machinery is a practical architectural choice that reduces dependence on external network backends for core orchestration.

The strongest claims—zero state loss under continuous interpolation, isolation guarantees under multi-agent load, and definitive superiority over all discrete alternatives—remain authorial assertions pending the kind of deterministic, logged isolation tests outlined in Section 5. Design and structure evidence is already substantial; runtime verification of the continuous properties is the open engineering task that would convert architectural description into reproducible functional proof at the highest standard.

Positioning the work for public archival (e.g., GitHub) is best served by releasing: (1) this inventory and research paper, (2) the original whitepapers and monograph with version dates, (3) extracted critical modules with hashes, and (4) a minimal reproducible test suite that captures primary-versus-sandbox state and route/PAD transitions. Such a package allows third parties to evaluate the claims against runnable artifacts rather than documentation alone. Until runtime isolation logs are published, external readers should treat the continuous-interpolation and zero-loss claims as design goals supported by structure, not as independently demonstrated measurement results.

The local-first stance is a coherent response to real problems in cloud-centric multi-agent stacks: latency, privacy exposure, and brittle context. Whether the continuous DS formalization delivers measurable advantages under load is an empirical question that the recommended verification protocol is designed to answer. The existence of another project using similar terminology does not diminish the substance of the Qyvaria design or the volume of engineering work visible in the repository scans.

8.1 Limits of the Present Evidence Base

This paper does not certify performance numbers, does not claim physical dimensional effects, and does not treat terminology similarity as code theft. It does document a large, consistent design corpus and the presence of correspondingly named software components. Readers seeking the strongest possible functional claim should prioritize the isolation-test appendix recommended above over additional prose claims.

9. Conclusion

Qyvaria implements and documents a local-first modular AI operating kernel centered on a single-file Python runtime and a computational Dimensional Shift architecture defined by continuous state interpolation, sandboxed lookahead (MetaSimAgent), dynamic paradigm routing (BitWeaver / OnePy), and multi-axis behavioral state (PAD). Design documentation is extensive and consistent. Codebase structure evidence confirms presence of the named components across the single-file bundle and a large supporting repository. Runtime demonstration of every claimed continuity and isolation property under controlled multi-agent conditions is not yet established at the level of the strongest assertions. The work is the author's engineering product; independent projects that reuse related terminology do not, on the evidence examined, own or subsume the Qyvaria implementation. Further deterministic verification will strengthen the public case for functional claims while the existing design and structural corpus already documents a substantial original effort suitable for archival release as proof of design and engineering.

10. Source Basis and References (Supplied Materials)

Havlasek, J. Shifting Dimensions Architecture: Technical Monograph & Comparative Proof of Execution. Qyvaria Core. Status: Verified Functional Standard (author designation).

Havlasek, J. Qyvaria System Specification: Proof of Dimensional Shift Mechanics Across Ecosystem Implementations. Formal Proof / Architectural Standard.

Havlasek, J. Qyvaria: Mechanics of Continuous Dimensional Shifting. Architectural Specification & System Validation.

Havlasek, J. Qyvaria Ecosystem: Architecture of a Local-First Modular AI Operating Kernel. Qyvaria AI Systems & Cybernetics Group, Prague, Czech Republic.

Qyvaria Dimensional Shift — Evidence Report (source-grounded technical analysis of qyvaria.py bundle and architecture materials). Distinguishes design, structure, and runtime evidence levels.

Technical Analysis Report: qyvaria.py Execution Architecture (custom import hooks find_spec/create_module/exec_module, memory virtualization, messaging hub, self-contained bootstrapping).

DS_SCAN_REPORT.txt — Keyword and class matches for BitWeaver and MetaSimAgent inside qyvaria.py virtual modules.

QYVARIA_EXHAUSTIVE_AUDIT_REPORT.md / Qyvaria Ecosystem Targeted System Audit Report — Repository scan: 3,233 files, 1,992 matched modules, 387,963 keyword matches.

qyvaria_readme.py utility and bundle inventory metadata (OnePy Fusion++ single-file format, module lists, SHA-256 integrity fields).

DimensionalShift_Academic_Complete_vFinal.zip (Ji■í Burda; independent academic package; referenced for terminology and architecture comparison only).

Appendix A. Formal Mathematical Summary

Primary state: S ∈ ■^D (high-dimensional memory / representation space).
Compressed target: Ω ∈ ■^d with d ■ D.
Interpolation parameter: α(t) ∈ [0, 1], evolving continuously across processing ticks.
Transformation: Ψ(t) = (1 – α(t)) · Q(S_raw) + α(t) · T(S_compressed).
Intent: gradient flow without state snapping; smooth lookahead and behavioral recalculation.
Illustrative continuous projection pattern (author materials): base_projection = tensor_stream * (1.0 – alpha_shift); shifted_projection = (tensor_stream @ manifold_weights) * alpha_shift; return base_projection + shifted_projection.

Appendix B. Subsystem Mapping (Continuous vs Discrete)

Subsystem	Discrete Model (Static)	Qyvaria Continuous Model (Shifting)
MetaSimAgent	Hard-forked isolated sandboxes	Probabilistic lookahead timeline bleeding into main memory
Paradigm Router	Switch statement routing between engines	Unified routing fabric blending symbolic rules and vector spaces
BitWeaver Core	Fixed quantization passes	Dynamic bit-stream interleaving based on real-time entropy
PAD Coordinate Engine	Threshold-based emotion triggers	Continuous trajectory smooth-scaling over 3D manifold

Table 6. Subsystem mapping from continuous-shift whitepaper (author presentation).

Appendix C. Practical Verification Checklist

The following checklist is recommended for any future evidence appendix that aims to elevate runtime status from “not established” to “demonstrated under test”:

- Identify exact source file and function implementing the chosen shift path (e.g., MetaSimAgent.fork_subsim or equivalent).
- Define primary state serialization format (hashable snapshot or canonical byte representation).
- Define sandbox state serialization format.
- Record primary state hash before call.
- Invoke shift path with fixed seed / fixed inputs.
- Record primary state hash after call; assert equality (or tolerance) with pre-call hash.
- Record sandbox state; assert inequality with primary (or expected divergence metric).
- Record paradigm route identifier and/or PAD vector before and after; assert expected change or stability according to test design.
- Emit machine-readable log (JSON or structured text) with timestamps, inputs, hashes, and assertion results.
- Attach log and minimal reproducing script to the public archive.

Appendix D. Extended Inventory Notes from Scans

The exhaustive audit and DS scan materials collectively indicate: presence of OnePy Fusion++ single-file AI bundle header and __BUNDLE__ metadata; MetaSimAgent virtual module with class definition and command registry (compile, run, traces, plan, fork_subsim, snapshot, load, config); BitWeaver virtual module with class definition and CLI-style operations (init, ingest, search, reconstruct); supporting simulation and agent modules (SimulationInterface, WorldModel, aeon continuum variants); custom import machinery; Electron dashboard services for local engine control and diagnostics; model manifests for multiple Qwen-family persona layers. Keyword match volume across the tree is high (hundreds of thousands of instances in the exhaustive report), consistent with extensive naming of Qyvaria-related symbols, agents, and services throughout the codebase. These notes reinforce structural presence; they do not substitute for the isolation tests of Appendix C.

Appendix E. Archival Release Recommendation

For GitHub or similar public archival as proof of work, a minimal strong package would include: (1) this research paper; (2) the original whitepapers and comparative monograph with clear version/date markers; (3) extracted source of MetaSimAgent, BitWeaver, import-hook layer, and routing entry points with SHA-256 hashes; (4) a runnable isolation test script and its log output following Appendix C; (5) a short

README that states explicitly what has been demonstrated versus what remains design-level. Such a package maximizes credibility by separating demonstrated behavior from architectural intent while still showcasing the full scope of the engineering effort.

End of Research Paper. All claims are bounded by the supplied documents and scan artifacts. No physical dimensional claims are made. Runtime isolation and continuity under the strongest assertions require further deterministic demonstration. Generated 2026-10-25 12:41 CET for archival and proof-of-work use.

Author of the software systems under analysis: John Havlasek. This document is a source-grounded technical inventory and research presentation of those systems. It is intended for GitHub archival release as documentation of design, structure, and the current boundary of runtime verification.